

Topic 1: Merge Sort

Question-1

```
#include <stdio.h>
```

```
void merge (int arr[], int l, int m, int r)
```

```
{
```

```
    int n1 = m - l + 1;
```

```
    int n2 = r - m;
```

```
    int L[n1], R[n2];
```

```
    for (int i = 0; i < n1; i++)
```

```
        L[i] = arr[l + i];
```

```
    for (int i = 0; i < n2; i++)
```

```
        R[i] = arr[m + 1 + i];
```

```
    int i = 0, j = 0, k = l;
```

```
    while (i < n1 & j < n2)
```

```
    {
```

```
        if (L[i] <= R[j])
```

```
            arr[k++] = L[i++];
```

```
        else
```

```
            arr[k++] = R[j++];
```

```
    }
```

```
    while (i < n1)
```

```
        arr[k++] = L[i++];
```

```
    while (j < n2)
```

```
        arr[k++] = R[j++];
```

```
}
```

```
void mergesort (int arr[], int l, int r)
{
    if (l < r)
    {
        int m = (l+r)/2;
        mergesort (arr, l, m);
        mergesort (arr, m+1, r);
        merge (arr, l, m, r);
    }
}
```

```
int main()
{
    int n;
    printf ("Enter no. of students : ");
    scanf ("%d", &n);
```

```
int arr[n];
```

```
printf ("Enter the marks: \n");
for (int i=0; i<n; i++)
    scanf ("%d", &arr[i]);
```

```
mergesort (arr, 0, n-1);
```

```
printf ("Marks in ascending order: \n");
for (int i=0; i<n; i++)
    printf ("%d", arr[i]);
```

```
return 0;
```

```
}
```

Question - 2

```
#include <stdio.h>
```

```
void merge (int price[], int low, int mid, int high)
```

```
{
```

```
    int i, j, k;
```

```
    int n1 = mid - low + 1;
```

```
    int n2 = high - mid;
```

```
    int left[n1], right[n2];
```

```
    for (i = 0; i < n1; i++)
```

```
        left[i] = price[low + i];
```

```
    for (j = 0; j < n2; j++)
```

```
        right[j] = price[mid + 1 + j];
```

```
    i = 0;
```

```
    j = 0;
```

```
    k = low;
```

```
    while (i < n1 && j < n2)
```

```
    {
```

```
        if (left[i] <= right[j])
```

```
        {
```

```
            price[k] = left[i];
```

```
            i++;
```

```
        }
```

```
    else
```

```
    {
```

```
        price[k] = right[j];
```

```
        j++;
```

}

k++;

}

while(i < n/2)

{

price[k] = left[i];

i++;

k++;

}

while(j < n/2)

{

price[k] = right[j];

j++;

k++;

}

}

void mergesort(int price[], int low, int high)

{

if(low < high)

{

int mid = (low + high) / 2;

mergesort(price, low, mid);

mergesort(price, mid + 1, high);

merge(price, low, mid, high);

}

}

int main()

{

int n, i;

```
printf ("Enter number of products : ");  
scanf ("%d", &n);
```

```
int price [n];
```

```
printf ("Enter product prices : \n");
```

```
for (i=0; i<n; i++)  
{
```

```
    scanf ("%d", &price[i]);  
}
```

```
mergesort (price, 0, n-1);
```

```
printf ("\n Prices in ascending order: \n");
```

```
for (i=0; i<n; i++)  
{
```

```
    printf ("%d", price[i]);  
}
```

```
printf ("\n");
```

```
return 0;
```

```
}
```

Ques 3:

#include <stdio.h>

void Merge (int salary [], int left, int mid, int right)

{

int i, j, k;

int n1 = mid - left + 1;

int n2 = right - mid;

int L[n1], R[n2];

for (i = 0; i < n1; i++)

L[i] = salary [left + i];

for (j = 0; j < n2; j++)

R[j] = salary [mid + 1 + j];

i = 0;

j = 0;

k = left;

while (i < n1 && j < n2)

{

if (L[i] <= R[j])

{

salary[k] = L[i];

i++;

}

else

{

```
salary[k] = R[j];
```

```
j++;
```

```
k++;
```

```
}
```

```
}
```

```
void mergesort (int salary[], int left, int right)
```

```
{
    if (left < right)
    {
```

```
        int mid = left + (right - left) / 2;
```

```
        mergesort (salary, left, mid);
```

```
        mergesort (salary, mid + 1, right);
```

```
        merge (salary, left, mid, right);
```

```
    }
```

```
}
```

```
int main()
```

```
{
```

```
    int n, i;
```

```
    printf ("Enter no. of employees: ");
```

```
    scanf ("%d", &n);
```

```
    int salary[n];
```

```
    printf ("Enter the salaries: ");
```

```
    for (i = 0; i < n; i++)
```

```
    {
```

```
        scanf ("%d", &salary[i]);
```

```
    }
```

```
    mergesort (salary, 0, n - 1);
```

```
    printf ("n Employee Salaries in ascending  
order: ");
```

```
for (i=0; i<n; i++)
```

```
{  
    printf("%d", salary[i]);  
}
```

```
printf("\n");
```

```
printf("\n Lowest salary = %d", salary[0]);  
printf("\n Highest salary = %d", salary[n-1]);  
printf("\n Difference = %d", salary[n-1] - salary[0]);
```

```
return 0;
```

```
}
```

Ques 4

```
#include <stdio.h>
```

```
int partition (int time [], int low, int high)
{
```

```
    int pivot = time [high];
```

```
    int i = low - 1;
```

```
    int j, temp;
```

```
    for (j = low; j < high; j++)
    {
```

```
        if (time [j] < pivot)
        {
```

```
            i++;
```

```
            temp = time [i];
```

```
            time [i] = time [j];
```

```
            time [j] = temp;
```

```
        }
```

```
    }
```

```
    temp = time [i+1];
```

```
    time [i+1] = time [high];
```

```
    time [high] = temp;
```

```
    return i+1;
```

```
}
```

```
void quicksort (int time [], int low, int high)
{
```

```
    if (low < high)
    {
```

```
        int loc = partition (time, low, high);
```

```

    quicksort (time, low, loc - 1);
    quicksort (time, loc + 1, high);

```

```

}

```

```

{

```

```

    int main()

```

```

    {

```

```

        int n, i;

```

```

        printf("Enter no. of runners: ");

```

```

        scanf("%d", &n);

```

```

        int time [n];

```

```

        printf("Enter race timings: \n");

```

```

        for (i = 0; i < n; i++)

```

```

        {
            scanf("%d", &time[i]);

```

```

        }

```

```

        quicksort (time, 0, n - 1);

```

```

        printf("\n Race Timings in Ascending Order: \n");

```

```

        for (i = 0; i < n; i++)

```

```

        {
            printf("%d", time[i]);

```

```

        }

```

```

        printf("\n");

```

```

        printf("\n Fastest Runner Time = %d", time[0]);

```

```

        printf("\n Slowest Runner Time = %d", time[n - 1]);

```

```

        return 0;

```

```

    }

```

Ques 5

```
#include <stdio.h>
```

```
int partition(int bookID[], int low, int high)
```

```
{
```

```
    int pivot = bookID[high];
```

```
    int i = low - 1;
```

```
    int j, temp;
```

```
    for (j = low; j < high; j++)
```

```
    {
```

```
        if (bookID[j] <= pivot)
```

```
        {
```

```
            i++;
```

```
            temp = bookID[i];
```

```
            bookID[i] = bookID[j];
```

```
            bookID[j] = temp;
```

```
        }
```

```
    }
```

```
    temp = bookID[i+1];
```

```
    bookID[i+1] = bookID[high];
```

```
    bookID[high] = temp;
```

```
    return i+1;
```

```
}
```

```
void quicksort(int bookID[], int low, int high)
```

```
{
```

```
    if (low < high)
```

```
    {
```

```
        int p = partition(bookID, low, high);
```

```

        quicksort (bookID, low, p-1);
        quicksort (bookID, p+1, high);
    }
}

int main()
{
    int n, i;
    printf ("Enter number of books: ");
    scanf ("%d", &n);

    int bookID[n];
    printf ("Enter book IDs: \n");

    for (i=0; i<n; i++)
    {
        scanf ("%d", &bookID[i]);
    }

    quicksort (bookID, 0, n-1);

    printf ("\n Book IDs in ascending order: \n");
    for (i=0; i<n; i++)
    {
        printf ("%d", bookID[i]);
    }
    printf ("\n");

    return 0;
}

```

Ques 6.

```
#include <stdio.h>
```

```
int partition (int sales[], int low, int high)
```

```
{
```

```
    int pivot = sales [high];
```

```
    int i = low - 1;
```

```
    int j, temp;
```

```
    for (j = low; j < high; j++)
```

```
    {
```

```
        if (sales [j] <= pivot)
```

```
        {
```

```
            i++;
```

```
            temp = sales [i];
```

```
            sales [i] = sales [j];
```

```
            sales [j] = temp;
```

```
        }
```

```
    }
```

```
    temp = sales [i + 1];
```

```
    sales [i + 1] = sales [high];
```

```
    sales [high] = temp;
```

```
    return i + 1;
```

```
}
```

```
void quicksort (int sales[], int low, int high)
```

```
{
```

```
    if (low < high)
```

```
    {
```

```
        int p = partition (sales, low, high)
```

```

    quicksort (sales, low, p-1);
    quicksort (sales, p+1, high);
}

```

```

}
int main()
{

```

```

    int n, i;
    float average;

```

```

    printf("Enter no. of sellers: ");
    scanf("%d", &n);

```

```

    int sales[n];

```

```

    printf("Enter sales of each seller: \n");

```

```

    for (i=0; i<n; i++)
    {

```

```

        scanf("%d", &sales[i]);
    }

```

```

    quicksort(sales, 0, n-1);

```

```

    printf("\n sales ranking : \n");

```

```

    for (i=0; i<n; i++)
    {

```

```

        printf("%d", sales[i]);
    }

```

```

    printf("\n");

```

```

    if (n >= 3)
    {

```

```
average = (sales[0] + sales[1] + sales[2]) / 3.0;  
printf("In Top 3 sales: %.d%.d%.d ", sales[0],  
sales[1], sales[2]);  
printf("In Average of Top 3 sales = %.2f", average);  
}
```

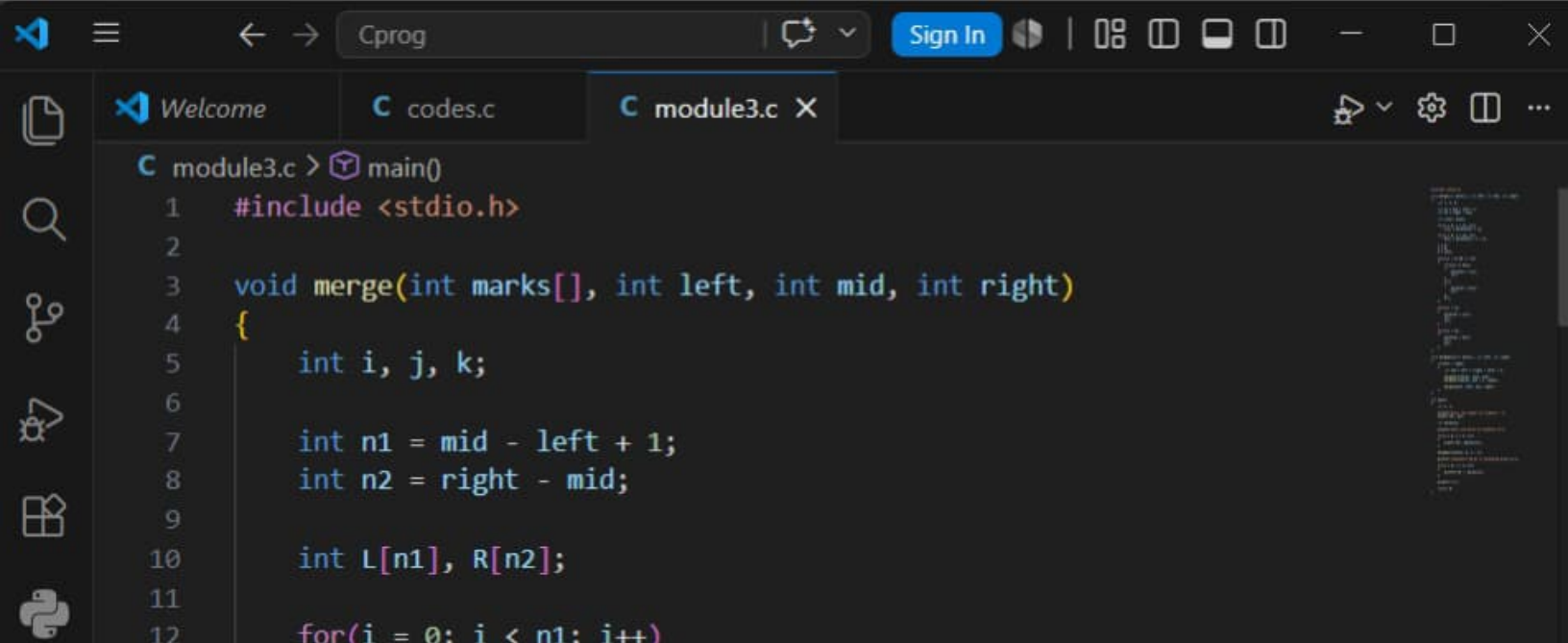
else

{

} printf("Not enough sellers to calculate avg.");

return 0;

}



```
PS C:\Users\Unnati\Desktop\Cprog> cd "c:\Users\Unnati\Desktop\Cprog\" ; if ($?) { gcc module3.c -o module3 } ; if ($?) { .\module3 }
```

Enter the number of students: 6

Enter the marks of students:

78

45

92

65

51

89

Students Marks in Ascending Order:

45 51 65 78 89 92

PS C:\Users\Unnati\Desktop\Cprog>

```

C module3.c > main()
1  #include <stdio.h>
2
3  void merge(int price[], int low, int mid, int high)
4  {
5      int i, j, k;
6
7      int n1 = mid - low + 1;
8      int n2 = high - mid;
9
10     int left[n1], right[n2];
11
12     for(i = 0; i < n1; i++)
13         left[i] = price[low + i];
14
15     for(j = 0; j < n2; j++)
16         right[j] = price[mid + 1 + j];
17
18     i = 0;
19     j = 0;
20     k = low;

```

PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS

Code
+
-

X

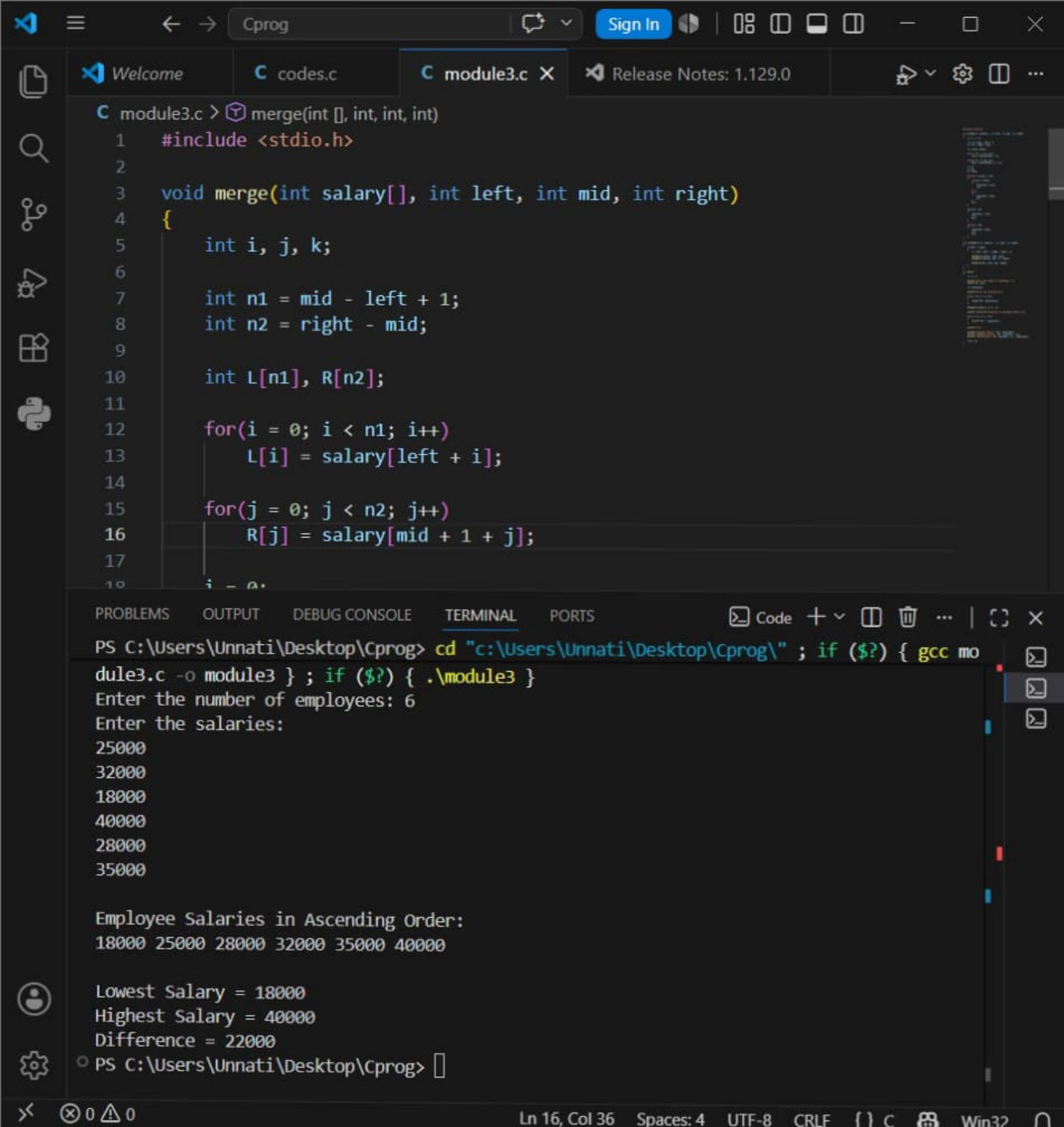
```

PS C:\Users\Unnati\Desktop\Cprog> cd "c:\Users\Unnati\Desktop\Cprog\" ; if ($?) { gcc mo
dule3.c -o module3 } ; if ($?) { .\module3 }
    for (i = 0; i < n1; i++)
    ~~~
• PS C:\Users\Unnati\Desktop\Cprog> cd "c:\Users\Unnati\Desktop\Cprog\" ; if ($?) { gcc mo
dule3.c -o module3 } ; if ($?) { .\module3 }
Enter the number of products: 5
Enter product prices:
450
299
700
150
520

Prices in Ascending Order:
150 299 450 520 700
PS C:\Users\Unnati\Desktop\Cprog>

```

Ln 92, Col 14
Spaces: 4
UTF-8
CRLF
{} C
Win32



```

C module3.c > partition(int [], int, int)
1  #include <stdio.h>
2
3  int partition(int time[], int low, int high)
4  {
5      int pivot = time[high];
6      int i = low - 1;
7      int j, temp;
8
9      for(j = low; j < high; j++)
10     {
11         if(time[j] < pivot)
12         {
13             i++;
14
15             temp = time[i];
16             time[i] = time[j];
17             time[j] = temp;
18         }
19     }

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

Difference = 22000
PS C:\Users\Unnati\Desktop\Cprog> cd "c:\Users\Unnati\Desktop\Cprog\" ; if ($?) { gcc mo
dule3.c -o module3 } ; if ($?) { .\module3 }
Enter the number of runners: 6
Enter race timings:
52
45
60
48
55
50

Race Timings in Ascending Order:
45 48 50 52 55 60

Fastest Runner Time = 45
Slowest Runner Time = 60

```